

Introduction to Functions in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

This session aimed to build a solid foundation in C++ functions. The following concepts were covered:

- Understanding the concept and purpose of functions in C++
- Declaring function prototypes and writing function definitions
- Passing parameters to functions and handling return values
- Structuring programs as modular, reusable blocks

2 Theoretical Background

What is a Function?

A **function** is a named, self-contained block of code that performs a specific task. Functions let a large program be broken into manageable, reusable pieces. Each function has a name, can accept *parameters* as input, and can send a result back via a *return value*. Once written, a function can be called as many times as needed from anywhere in the program, eliminating repeated code and making programs shorter, more readable, and easier to debug.

Function Structure

A function definition has a *return type*, a name, a parameter list in parentheses, and a body in curly braces. When called, execution jumps to the body and returns to the call point when the function finishes. Functions are distinguished from variables by the parentheses following their name — e.g. `main()`.

Return Values

Every non-void function must return a value matching its declared return type using the `return` statement. A void return type tells the compiler no value will be sent back. The returned value can be stored, printed, or used directly in an expression.

Function Prototypes

A **prototype** tells the compiler a function's name, parameter types, and return type before its full body is provided. This allows the function to be called above its definition in the source file. Syntax: `returnType functionName(parameterTypes);`

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Function Introduction and Declaration

A void function named greet takes no parameters. When called, it prints a welcome message, demonstrating the basic pattern of defining and invoking a function.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
void greet() {
    cout << "Welcome to the C++ Programming Lab!" << endl;
}
int main() {
    greet();
    return 0;
}
```

Task 2 — Basic Function Declaration and Call

A void function named sayHello prints the classic Hello, World! message. Calling it from main() keeps the output logic neatly separated from program setup.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
void sayHello() {
    cout << "Hello, World!" << endl;
}
int main() {
    sayHello();
    return 0;
}
```

Task 3 — Array Sum and Average Using a Function

Ten integers are entered by the user. A function calculateSum accepts the array and its size, loops through elements, and returns the total. The average is computed in main() from the returned sum.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int calculateSum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) sum += arr[i];
    return sum;
}
int main() {
    int numbers[10];
    for (int i = 0; i < 10; i++) { cout << "Value " << i+1 << ": "; cin >> numbers[i]; }
    int total = calculateSum(numbers, 10);
    cout << "Sum = " << total << ", Avg = " << (double)total/10.0 << endl;
    return 0;
}
```

5 Conclusion

This lab introduced core function concepts in C++ — prototypes, definitions, and the calling mechanism. The tasks showed how separating code into functions keeps programs clean and reusable, how void functions handle output-only tasks, and how passing arrays to functions enables computations without duplicating logic. These modular design principles are essential for writing maintainable programs.